

Quantifying SZZ-Induced Label Noise in Just-In-Time Defect Prediction

A Comprehensive Synthesis — What, How, and Why

Author	Puneet Deshwani — M.Tech Thesis
Status	Phases 1 and 2 complete and validated · Phases 3 and 4 designed and implemented, not yet executed
Corpus	21 Apache Java projects · 27,319 commits · 2,332 labelled defect-introducing (8.54%)
Evidence base	16,380 evaluation records · 74 paired statistical tests · every number reproducible from committed CSVs

1. The Core Problem & Motivation

1.1 The problem in one sentence

Almost everything the software engineering community believes about automated defect prediction rests on training labels that were never verified, evaluated by procedures that cannot occur in practice — and nobody had measured how much those two facts inflate the reported results.

1.2 The setting, in plain terms

When a developer commits code, some of those commits introduce bugs. **Just-In-Time Software Defect Prediction (JIT-SDP)** trains a machine learning model to flag risky commits at the moment they are made, so reviewers can look harder at them.

To train such a model you need labelled examples: commits known to have introduced a defect. But nobody maintains such a list. So the field infers it with an algorithm called **SZZ**, which works backwards:

1. Find a commit that fixed a bug.
2. Look at which lines that fix changed.
3. Use `git blame` to find which earlier commit last touched those lines.
4. Declare that earlier commit “defect-introducing.”

Every step is a guess. Step 2 is the weakest: a bug-fix commit usually also contains renaming, reformatting, comment edits and unrelated tidying — what the literature calls **tangled commits**. SZZ blames the authors of *all* those lines, not just the ones that fixed the bug.

1.3 Two unexamined assumptions

The field has operated on two assumptions that had never been jointly tested:

The Label Assumption — that SZZ output is accurate enough to serve as ground truth for both training *and* evaluation.

The Evaluation Assumption — that standard machine-learning evaluation (random cross-validation, immediately available labels) reflects what a deployed system would achieve.

Both are false in ways that compound, and the compounding had not been quantified.

1.4 Why the evaluation assumption fails

Two mechanisms:

Temporal leakage. Random k-fold cross-validation shuffles commits, so a model can learn from 2019 commits to predict 2015 commits. In deployment the future is not available. Any model capable of memorising project-specific temporal patterns will look better than it is.

Verification latency. In reality you do not learn that a commit was buggy until someone finds and fixes the bug — which takes time. Measured on this corpus: **median 113 days, 90th percentile 1,597 days, and 53% of defect labels arrive after a 90-day decision window.** A deployed system spends most of its life training on labels it will later discover were wrong.

1.5 Why the problem is significant

It is foundational, not incremental. If the labels and the evaluation are both compromised, then a decade of reported improvements may be measuring label-reproduction skill rather than bug-finding skill. Every subsequent result inherits the error.

It has a practical cost. Organisations deploy these tools to allocate scarce review effort. A model reported at MCC 0.40 that actually delivers 0.10 misdirects that effort.

It is measurable, and nobody had measured it. Prior work studied SZZ noise *or* evaluation realism, separately. This thesis isolates and quantifies both, and their interaction, on one corpus with one protocol.

1.6 The research questions

RQ	Question	Status
RQ1	How much do SZZ variants disagree with each other and with a higher-quality reference, and in which direction does each variant err?	Answered (Phase 1)
RQ2	How does the choice of label source shift measured performance across increasingly honest evaluation regimes?	Answered (Phase 2)
RQ3	Through what mechanism does label noise degrade an online learner as noise dose increases?	Designed (Phase 3)
RQ4	Can modulating an online learner's oversampling by per-instance label confidence recover the lost performance?	Designed (Phase 4)

2. Step-by-Step Methodology & Execution

2.1 The reference labels, and exactly what they are

This deserves precision, because the entire comparison rests on it.

The reference (“oracle”) labels come from **JIT-Defects4J** (Ni et al., ESEC/FSE 2022), published on Zenodo as part of the JIT-Fine replication package, and built as an extension of **LLTC4J** (Herbold et al.).

The construction has two stages, and only the first is human:

Stage	Method	Verified?
Which lines in a bug-fix genuinely fix the bug	Human annotation — at least three participants agreeing	Yes
Which commit introduced those lines	git blame	No — algorithmic
Which commits are clean	Everything not flagged above	No — by residual

Consequence, stated plainly: the oracle is *not* manually verified ground truth for defect-introducing commits. It is **git blame seeded with human-verified fix lines**.

This matters enormously for interpretation. SZZ blames every line touched in a fix; the oracle blames only the lines annotators agreed were fixing the bug; **both then use the same git blame step**. So every comparison in this thesis isolates one specific thing — **the cost of tangled commits** — and not blame error, which sits on both sides and cancels.

Provenance was verified rather than assumed: the paper’s dataset table matches this corpus project by project (commons-vfs 114/1110, giraph 163/844, gora 39/553, opennlp 91/1086, parquet-mr 158/1120; total 2,332/27,319 = 8.54%).

2.2 Data collection

Feature data. Fourteen change-level features defined by Kamei et al. — diffusion (`ns` , `nd` , `nf` , `entropy`), size (`la` , `ld` , `lt`), purpose (`fix`), history (`ndev` , `age` , `nuc`) and developer experience (`exp` , `rexp` , `sexp`) — extracted from the JIT-Fine package for all 27,319 commits.

SZZ labels. Six variants were run with **PySZZ v2** over locally cloned copies of all 21 repositories, spanning the naive-to-conservative spectrum:

Variant	Character
BSZZ	Basic — blames every line modified in the fix
AGSZZ	Annotation-graph refinement
MASZZ	Meta-change aware (filters refactorings)
LSZZ	Line-number mapping, most conservative
RSZZ	Restricted / refined
RASZZ	Refactoring-aware

Verification timestamps. For every defect-introducing commit, the date its label would actually have become available: the author date of the earliest linked bug-fix commit. Reconstructed from the 5,453 Defects4J fix hashes plus **git log** across the cloned repositories.

2.3 Data preprocessing

Feature normalisation. Signed log compression, $\text{sign}(x) \cdot \log(1+|x|)$, applied to all features. Commit features are heavily right-skewed — a handful of commits change thousands of lines — and raw values dominate tree splits and gradient steps alike.

Label alignment. All six SZZ label sets were merged onto the feature table keyed on (`project` , `commit_id`) with a left join over the oracle universe. Commits for which a variant emitted no determination were set to 0, i.e. “SZZ made

no claim, therefore not flagged” — the semantics any real consumer of SZZ output would adopt.

Class imbalance. The corpus is 8.54% defective. Batch models use SMOTE oversampling plus a tuned decision threshold; the online learner uses Poisson-weighted oversampling internally.

Chronological ordering. Every project is sorted by author timestamp before any split. This is a precondition for honest evaluation, not a preprocessing nicety.

2.4 How the inputs were structured

Each experimental cell is one point in a four-dimensional grid:

Dimension	Levels
Model	LApredict · JITLine · ORB
Training label source	oracle + 6 SZZ variants (7)
Evaluation regime	naive k-fold · chronological · chronological-online · prequential-with-latency
Scoring convention	oracle-scored · self-scored

The scoring convention is the conceptual heart of the design.

- **Self-scored** — the model is evaluated against the same noisy labels it trained on. This is what the literature does. It measures *how well the model reproduces the heuristic*.
- **Oracle-scored** — the model is evaluated against the held-out reference labels. It measures *how well the model finds actual defects*.

The gap between them is the quantity nobody had isolated.

2.5 The four evaluation regimes, from dishonest to realistic

1. Naive k-fold. Random 10-fold cross-validation. Deliberately included as the dishonest baseline, because it is what much of the literature used.

2. Chronological. Train on the earliest 50% by time, test on the latest 50%. Removes temporal leakage.

3. Chronological-online. The online learner consumes the first half sequentially, is then frozen, and predicts the second half. Added specifically to hold model architecture constant while changing regime.

4. Prequential with verification latency. Full streaming simulation. For each commit in time order: the model predicts, is scored, and only later receives its label on the real schedule — clean labels after the 90-day window, defect labels at the actual fix date. Critically, **a defect whose fix arrives after the window is first delivered as a wrong “clean” label and corrected later**, reproducing the false-negative noise a deployed system genuinely experiences.

2.6 Execution and reproducibility

Scale. 21 projects × 10 random seeds × 7 label sources × models × regimes = **16,380 evaluation records**.

Where it ran. The full grid runs on GitHub Actions (~3–5 hours). The local machine has 7.1 GB RAM and no swap, and a memory-heavy extraction step froze it once during development — after which the two bulky inputs (a 101 MB archive and 830 MB of cloned repositories) were replaced by two small committed derived files (4.2 MB and 0.37 MB), making the whole pipeline runnable on a CI runner from small inputs.

An integrity incident worth recording. Mid-project it emerged that the Phase 2 dataset cache had gone stale relative to a Phase 1 label regeneration: models had trained on labels one commit older than the noise rates being reported, affecting 1.0–6.5% of commits per variant. Root cause was an unguarded cache. The fix was three-part — a content-

hash staleness guard, a provenance sidecar recording the SHA-256 of every label file, and an automated consistency gate that runs in CI *before any compute is spent*. Verified against the stale data, the gate fails on all six variants; against corrected data it passes.

This is worth stating in the thesis rather than hiding: a project about label-provenance fragility encountered exactly that fragility in its own pipeline, and now ships an automated assertion against it.

3. Architectural Decisions & The “Why”

3.1 Model selection

Three models were chosen to span the capacity spectrum, because **capacity turned out to be the variable that governs how much evaluation dishonesty inflates a result**.

Model	What it is	Why included
LApredict	Logistic regression on a single feature — lines added	A deliberately minimal baseline. Cannot memorise, so it acts as a control: any inflation it shows is a property of the data, not the model.
JITLine	100-tree Random Forest on all 14 features, SMOTE, tuned threshold	Representative high-capacity batch learner, the realistic target of the inflation critique.
ORB	Online ensemble of 20 incremental logistic regressors with Poisson oversampling and prediction-bias boosting	The canonical streaming learner designed for verification latency (Cabral et al., 2019) — the only one that can be evaluated under realistic label arrival.

Why not deep models (DeepJIT, CC2Vec)? Recent benchmarks show they rarely beat simple baselines once time-aware splits are enforced, and Finding 1 below explains the mechanism: inflation scales with capacity, so deep models were the *most* inflated. Adding them would have added compute and another confound without changing the argument.

Instrument validation. Before drawing any conclusion from ORB, it was replicated against Cabral et al.’s own 14 datasets: G-mean 0.46–0.93, mean ≈ 0.68 — squarely in the published range. This establishes that the low absolute scores on this corpus are a property of the data, not a broken implementation. (Caveat stated openly: those datasets run at 22–43% defect rates against this corpus’s 8.5%.)

3.2 Metric selection

MCC (Matthews Correlation Coefficient) as primary. It is chance-anchored at zero and uses all four confusion-matrix cells, so it cannot be inflated by majority-class prediction. At 8.5% positives, accuracy and F1 are both actively misleading.

G-mean (geometric mean of per-class recalls) as secondary, to expose models that achieve MCC by collapsing onto one class.

Precision, recall, ρ_0 (false-alarm rate) and ρ_1 (miss rate) for Phase 1 label quality — ρ_0 and ρ_1 specifically because they are the parameters a noise-injection experiment needs.

3.3 Three pivots, each forced by evidence

These are worth recording because each represents a claim that was tested and abandoned rather than assumed.

Pivot 1 — Decision threshold protocol. JITLine’s original protocol fit a forest on 80% of the training split, tuned the threshold on the remaining 20%, then refit on 100% and kept the stale threshold. The refit shifted calibration by roughly 0.35 in probability, producing predicted-positive rates from 1.4% to 68.5% across projects. Four protocols were then measured:

Mode	Oracle MCC	Oracle G-mean	Degenerate runs
tail (original)	0.1031	0.4855	9.5%
oob (out-of-bag)	0.0874	0.3655	17.5%
cv (blocked)	0.1004	0.5226	4.8%
fwd (forward-chaining — adopted)	0.1011	0.5235	4.8%

Out-of-bag tuning was the intuitive fix and measured *worst* — OOB probabilities come from only ~63% of the trees, so the threshold is miscalibrated against the full ensemble. Forward chaining was adopted because it matches blocked CV on every measure while only ever looking backwards, as deployment requires.

Pivot 2 — The prequential summary statistic. The streaming metric was originally the terminal value of a fading confusion matrix. With fading 0.99 that summarises roughly the last 100 commits — about 8 positives — and carries twice the project-level variance of averaging the whole trajectory. Switching estimators changed one conclusion outright. Neither summary is canonical (MCC is not a decomposable loss, so the standard prequential-with-fading construction does not extend to it), so **both are reported and the choice is justified by measured variance, not by citation.**

Pivot 3 — A withdrawn causal claim. An earlier analysis concluded that verification latency accounted for only ~9% of the batch-to-streaming performance drop. That comparison was **not identified**: the two regimes differed in three ways at once — frozen versus continually adapting, immediate versus delayed labels, and second-half versus whole-stream evaluation. The effects ran in opposite directions and cancelled. Matching only the evaluation window flips the sign of the original contrast. The claim was withdrawn and replaced with a proper 2×2 design.

3.4 Engineering Research Methods — the statistical protocol

Unit of analysis: the project (n = 21). Seeds are averaged first. Seeds are repeated measurements of the same project, not independent observations; treating 16,380 records as the sample size would be pseudo-replication and is the first thing a methodologist would attack.

Test: paired Wilcoxon signed-rank. Paired because the same 21 projects appear in both conditions, and between-project variance is enormous (project MCCs range -0.09 to +0.18). Non-parametric because MCC across projects is bounded, skewed, and outlier-prone.

Effect size: matched-pairs rank-biserial correlation, derived from the same signed ranks as the p-value, reported with the **Hodges–Lehmann** location estimate and a **percentile bootstrap 95% CI for that estimator** — the HL statistic recomputed inside every resample, so the interval targets the quantity it qualifies.

An earlier version reported Cliff’s delta computed all-versus-all, which discards the pairing the design preserves. The difference is not cosmetic: the headline comparison reads rank-biserial +1.000 paired against +0.74 unpaired.

Multiplicity: Holm–Bonferroni, reported twice. Once within test family, once globally across all 74 tests. Nothing was pre-registered — families were defined during analysis — so **every test is flagged exploratory and the global correction is provided as the conservative sensitivity that requires no argument about family definitions.**

Robustness rather than more tests. The estimator sensitivity grid (120 comparisons) is reported as a robustness analysis with grid-wide correction applied, not as 120 additional confirmatory findings.

4. Results and Performance Metrics

4.1 Phase 1 — How wrong is SZZ?

Every variant scored against the reference labels over an identical 27,319-commit denominator.

Variant	Precision	Recall	ρ_0 (false alarm)	ρ_1 (miss)
BSZZ	0.186	0.641	0.263	0.359
AGSZZ	0.186	0.467	0.192	0.533
MASZZ	0.183	0.482	0.201	0.518
LSZZ	0.272	0.267	0.067	0.733
RSZZ	0.232	0.300	0.093	0.700
RASZZ	0.184	0.438	0.181	0.562

Finding 1.1 — A precision ceiling of 27.2%. No variant exceeds it. Between 72.8% and 81.7% of everything SZZ flags disagrees with the reference.

Finding 1.2 — The noise is asymmetric and bifurcated, not random. BSZZ is false-positive-heavy ($\rho_0 = 26.3\%$, recall 64%). LSZZ and RSZZ are false-negative-heavy ($\rho_0 \approx 7\text{--}9\%$ but missing 70–73% of defects). This is why Phase 3’s noise injection must be calibrated per variant rather than using uniform random flips.

Finding 1.3 — The misses come from the refinements, not from blame. It is tempting to explain a 70–73% miss rate structurally: line-tracking cannot blame code that was never written. That explanation is unavailable here, because the reference labels are themselves `git blame` output, so every reference positive is blame-reachable by construction. Decomposing the false negatives against BSZZ instead:

Variant	Total FN	Also missed by BSZZ	Found by BSZZ, then filtered away
AGSZZ	1,242	765 (62%)	477 (38%)
MASZZ	1,208	763 (63%)	445 (37%)
RASZZ	1,310	765 (58%)	545 (42%)
RSZZ	1,633	808 (49%)	825 (51%)
LSZZ	1,710	807 (47%)	903 (53%)

Two mechanisms of similar weight: **seed-line divergence** (both procedures blame, but from different lines, and land on different commits) and **self-inflicted filtering** (the variant’s own refinement discards a commit BSZZ had correctly found). For the most conservative variants the second dominates — LSZZ discards 60.4% of the reference defects BSZZ had already identified. The refinements succeed at cutting false alarms from 26.3% to 6.7%, but over half of what they remove is correct.

Finding 1.4 — Variants agree with each other far more than with the reference. AGSZZ, MASZZ and RASZZ agree pairwise at $\kappa = 0.859\text{--}0.933$, but with the reference at only $\kappa = 0.158\text{--}0.202$. **They share failure modes.** High inter-tool agreement in the literature was mistaken for accuracy.

4.2 Phase 2 — What that noise costs

The inflation ladder

Step	Configuration	MCC
0	JITLine, BSZZ labels, k-fold, self-scored — the literature’s number	0.4129
1	Same model, same labels, oracle-scored	0.1751
2	Oracle labels, chronological split	0.1016
3	Online learner, prequential + real latency	0.0970

Presented as descriptive only. Step 3 changes the learner, the regime and the evaluation window simultaneously and must not be read as a causal decomposition — see §4.5.

Result 1 — Temporal leakage inflates, in proportion to model capacity

Model	k-fold	Chronological	HL	95% CI	rank-biserial	Global Holm
JITLine	0.2430	0.1016	+0.1373	[+0.105, +0.172]	+1.000	7.1e-05
LApredict	0.2058	0.1734	+0.0339	[−0.000, +0.086]	+0.489	1.000

rank-biserial +1.000 means the effect held in 21 of 21 projects with no exceptions — which is why the exact test saturates at its floor of $2/2^{21} = 9.5\text{e-}07$.

Why the asymmetry: a 14-feature Random Forest can memorise project-specific temporal patterns that shuffled folds expose. A one-feature logistic regression cannot memorise anything. **The inflation is a function of model capacity, not of the data** — which explains why weak baselines look competitive in the literature: they were never the ones being inflated.

All fourteen comparisons (7 label sources $\times$ 2 models) point the same direction; the universality was invisible when only three hand-picked sources were tested.

Result 2 — Self-scoring inflates more, and it is model-dependent

JITLine trained on BSZZ, under k-fold: **0.4129** scored against BSZZ, **0.1751** scored against the reference. Gap **HL +0.2303**, CI [+0.189, +0.278], rank-biserial +0.991, global Holm 1.3e-04.

This is the largest effect in the study.

Variant	JITLine gap	Significant?	LApredict gap	Significant?
BSZZ	+0.2303	Yes	+0.1549	Yes
MASZZ	+0.1985	Yes	+0.0689	No
RASZZ	+0.1861	Yes	+0.0598	No
AGSZZ	+0.1797	Yes	+0.0510	No
LSZZ	+0.1076	Yes	+0.0615	No
RSZZ	+0.0647	Yes	−0.0238	No

JITLine’s gap is significant for all six variants; LApredict’s only for BSZZ. Interpretation, stated as interpretation rather than demonstration: the gap is *compatible with* a high-capacity model fitting systematic structure in the heuristic’s

errors — structure a low-capacity model cannot represent. Demonstrating that mechanism would require directly testing whether SZZ’s false positives are predictable from the features, which has not been done.

Result 3 — Under realistic streaming, label quality matters

ORB under real verification latency, oracle-scored, time-averaged prequential MCC:

Training labels	MCC	HL vs oracle	95% CI	Global Holm
oracle	0.0970	—	—	—
LSZZ	0.0582	+0.0341	[+0.014, +0.055]	0.124
BSZZ	0.0578	+0.0412	[+0.022, +0.052]	0.006
RASZZ	0.0395	+0.0577	[+0.036, +0.081]	0.006
AGSZZ	0.0365	+0.0600	[+0.029, +0.085]	0.024
MASZZ	0.0352	+0.0638	[+0.039, +0.085]	0.004
RSZZ	0.0315	+0.0603	[+0.042, +0.082]	0.001

All six intervals exclude zero. Five of six survive global correction.

The sharpest reading of this result. Because the oracle and BSZZ share the identical `git blame` step and differ only in which lines seed it, the `oracle vs BSZZ` row is a near-controlled ablation of tangled commits:

Tangled commits cost +0.041 MCC (95% CI [+0.022, +0.052]) in streaming defect prediction.

That is a mechanism with an effect size and an interval attached — considerably stronger than “clean labels beat noisy labels.”

Result 4 — FP-heavy labels help batch learners, mechanism unresolved

Under chronological evaluation, BSZZ-trained JITLine (0.1324) *beats* oracle-trained JITLine (0.1016), winning in 15 of 21 projects.

The effect is real, not noise: **97.7%** of the between-project variance is genuine rather than seed variation, and 16 of 21 projects show a gap more than 2 standard errors from zero. It survives all four threshold protocols.

But the mechanism is not established. The natural explanation is class-imbalance relief — BSZZ labels 29.5% of commits positive against a true rate of 8.5%, giving the forest more minority mass. Against that: nine project-level predictors were tested (size, positive count, positive rate, training positives, enrichment ratio, precision, recall, coverage) and **none reaches significance**; and `opennlp` is the second-largest BSZZ win (+0.162) in a project where BSZZ flags a *lower* positive rate than the oracle (6.91% vs 8.38%), so augmentation cannot be the mechanism there.

The effect is also **batch-only**: under streaming, oracle beats BSZZ in 18 of 21 projects.

4.3 How each statistical test was performed, and what it establishes

Test family 1 — Regime inflation. For each model × label source, project-level mean MCC under k-fold is paired against the same project under a chronological split, and the 21 paired differences are tested. *Establishes*: evaluation protocol alone changes measured performance, and by how much.

Test family 2 — Self-deception gap. For each model × label source × regime, self-scored MCC is paired against oracle-scored MCC on the same project. *Establishes*: how much of reported performance is label reproduction rather than defect detection. Tested **per model** — an earlier version pooled two models into 42 pairs and described them as 21 projects, treating two models on one project as independent when they share the project, labels, features and split.

Test family 3 — Label source gap. ORB trained on oracle labels versus each SZZ variant, same regime, same projects. *Establishes:* whether label quality affects deployable performance.

Test families 4 and 5 — Batch/stream decomposition. Retired. They rested on a confounded contrast and no longer support a decomposition claim; they remain in the results file as descriptive comparisons, flagged as such.

Interpretation thresholds. $|\text{rank-biserial}|$ 0.1 small · 0.3 medium · 0.5 large. A p-value reports whether an effect exists; the effect size and interval report whether it matters. Both are always given.

4.4 Robustness — does the conclusion survive the analyst’s choices?

The streaming metric has two free parameters: which trajectory summary, and what fading factor. Both were varied systematically — fading 0.90 to 0.999 (effective windows **10 to 1000 commits**) crossed with warm-up skips of 0/5/10/25%, giving 20 settings × 6 variants.

Across all 120 comparisons	Result
Hodges–Lehmann estimates with oracle ahead	120 / 120
Bootstrap CIs excluding zero	120 / 120
Surviving grid-wide Holm correction	120 / 120

The label-source conclusion does not depend on the summary statistic, the fading factor, or the warm-up rule. Warm-up skipping raises oracle MCC monotonically, confirming that the early, under-trained stream *depresses* rather than inflates the average.

4.5 An honest negative result — latency could not be isolated

After withdrawing the flawed decomposition, a full 2×2 was run: continual adaptation crossed with label delay, every cell scored on an identical window.

MCC	Immediate labels	Delayed labels
Frozen	0.0827	0.0781
Adaptive	0.1268	0.1013

Contrast	HL	95% CI	p
Adaptivity, labels immediate	−0.0415	[−0.083, +0.002]	0.065
Adaptivity, labels delayed	−0.0146	[−0.044, +0.011]	0.393
Delay, adaptive learner	+0.0227	[−0.011, +0.052]	0.288
Delay, frozen learner	−0.0118	[−0.065, +0.047]	0.658
Interaction	−0.0296	[−0.073, +0.024]	0.338

Every interval contains zero. The delay penalty for an adaptive learner is estimated at about +0.02 MCC, and the interaction runs in the direction intuition suggests — a learner that has stopped updating cares less when labels arrive — but the data do not support either reading.

The correct statement is that 21 paired projects cannot resolve these effects. Resolving them requires more projects, not more seeds: seed-level variance is already small. This is reported as a limitation, not concealed as a gap.

5. Overall Significance & Conclusion

5.1 The ultimate takeaway

Most of the reported performance of just-in-time defect prediction is an artifact of how it is measured, and the largest single component is circular scoring — evaluating models against the same heuristic labels they were trained on.

Concretely, and all on one corpus with one protocol:

- Evaluating against the labels you trained on inflates measured performance by **+0.230 MCC**
- Random cross-validation inflates it by a further **+0.137 MCC**, and only for models with capacity to memorise
- Under realistic streaming with genuine verification latency, deployable performance is around **0.097 MCC**
- Controlling for tangled commits is worth **+0.041 MCC** [CI +0.022, +0.052]

5.2 How this advances the field

It converts a suspicion into a measurement. “SZZ is noisy” and “cross-validation leaks” were known. Neither had a number, a confidence interval, or a controlled contrast attached on a shared corpus.

It identifies model capacity as the moderator. Both inflation mechanisms scale with capacity. This explains an otherwise puzzling literature finding — that trivial baselines match sophisticated models — without appealing to dataset quirks. The sophisticated models were simply inflated more.

It names a mechanism rather than an association. Because the reference labels and BSZZ share an identical blame step and differ only in seed lines, the comparison isolates the cost of tangled commits specifically. That is a mechanistic claim with an interval, not a correlation.

It supplies a reusable protocol. Four evaluation regimes, dual scoring conventions, verification-latency simulation, project-level paired statistics with paired effect sizes and dual multiplicity correction — reusable by any subsequent JIT-SDP study.

It demonstrates methodological discipline, including where that hurt. Three claims were withdrawn or corrected on evidence: a causal decomposition that was not identified, an effect size that discarded the design’s pairing, and an appeal to a standard estimator that does not apply to this metric. A thesis that documents its own retractions is more trustworthy than one that reports only successes.

5.3 Practical implications

For researchers. Never evaluate on the same SZZ labels used for training — report the oracle-scored gap. Never use random k-fold on temporally ordered data. Always name the prequential summary statistic. Report paired effect sizes with intervals, and correct for multiplicity.

For practitioners. Expect roughly 0.10 MCC, not the 0.40 the literature suggests. Budget for verification latency: over half of defect labels arrive after three months, so a deployed model spends most of its life training on labels it will later find were wrong.

For dataset builders. Reducing tangled-commit noise is worth a measurable amount. Publishing the fix-to-inducing mapping alongside the labels would remove a construct-validity threat this thesis could not eliminate.

5.4 Threats to validity — stated plainly

The measured noise is a lower bound. The reference labels and SZZ share the same `git blame` step, so systematic blame error cancels. True noise against genuine ground truth would be larger by an unknown amount.

The corpus is SZZ-shaped by construction. The dataset authors discarded “*changes that do not add any new lines since the SZZ algorithm has an assumption that defects are introduced by adding new lines,*” so defects introduced purely by deleting code cannot appear at all. Separately, because the reference labels are themselves blame output, every reference positive is blame-reachable by definition — so blame-unreachability is not an available explanation for any measured false negative. Both constraints narrow what the corpus can speak to, and §4.1 reports the mechanisms that remain measurable on it.

Label-arrival timing is SZZ-derived. The reference labels have no native fix-to-inducing linkage in the published package, so arrival times were reconstructed from the union of SZZ mappings. A coverage sensitivity analysis (67.8% → 100% imputed) bounds part of this, but not the timing itself. The oracle is therefore blame-derived in construction and SZZ-derived in timing. **This is the central construct-validity threat.**

Statistical power. With 21 projects, effects smaller than roughly 0.04 MCC are not resolvable. The latency effect falls in that range and is reported as unresolved.

Project heterogeneity. Projects span 544–4,026 commits and 1.8%–19.3% defect rates, and are weighted equally. One project trains on five positive examples.

Single benchmark family. All 21 projects are Apache Java repositories from one dataset lineage.

5.5 Current status and what remains

Complete and validated: Phases 1 and 2 — the full label-quality characterisation and the downstream impact measurement, with all corrections applied and every number reproducible from committed data.

Implemented, not yet executed: Phase 3 (noise dose-response and mechanism diagnosis) and Phase 4 (Noise-Aware ORB, an online learner that modulates its oversampling by per-instance label confidence and provisionally rescues likely-delayed defect labels). Both runners are written, smoke-tested against the current codebase, and awaiting compute.

One finding already constrains Phase 3’s scope. Defects introduced by pure deletion are absent from the corpus by construction, and — more fundamentally — because the reference labels are themselves blame output, no false negative measured against them can be attributed to blame’s structural inability to trace a defect. Phase 3 can therefore study false-negative noise arising from **over-filtering** and from **seed-line divergence**, which §4.1 shows are the two mechanisms actually operating, but not from blame-unreachability. That limit should be declared before the experiment runs rather than discovered afterwards.

5.6 Closing

The thesis began by asking how much SZZ label noise costs a defect prediction model. The answer turned out to be less interesting than a question it exposed along the way: **how much of what the field reports is measurement artifact rather than capability?**

On this corpus, most of it. The literature’s 0.41 becomes 0.10 once the model is scored against something other than the heuristic that trained it, and evaluated in an order that time permits. That gap — roughly four-fifths of the reported signal — is the thesis’s central contribution, and it is now quantified, bounded by confidence intervals, corrected for multiple comparisons, robust across every analytical choice tested, and reproducible from a single commit hash.